Ants Algorithm
Advanced Computer Architecture

Anton Schelfhout

9 mei 2024



Instructor: A. Marinšek

1 Introduction

We developed an ant algorithm to find the shortest paths in graphs. We parallelised the algorithm on the GPU using CUDA and compared the impact of certain performance optimizations.

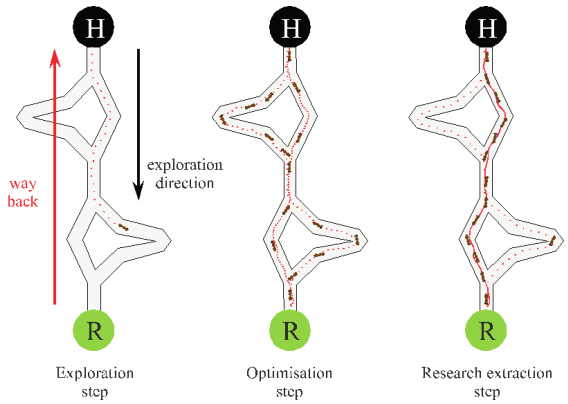
2 Ant Algorithm

The inspiration of an ant algorithm comes from ants in nature. When an ant finds a food source, it will walk back to its nest while releasing pheromones. The pheromones are followed by other ants which in return also release pheromones. In short, the more ants, the stronger the pheromones, leading to more ants. This concept is also called "stigmery", meaning two entities communicate not directly, but by changing their environment. In this way ants can very efficiently coördinate their colony.

We can apply this idea also to single source, shortest path algorithms. Each "ant" starts from the source node and chooses its next node based on the pheromone level on the edge leading to that node. This is done using a weighted probability function.
When an ant reaches the destination, it will walk back and leave its pheromones on the edge. The total amount of pheromones an ant can deposit is constant, so the shorter its path, the more pheromones per edge it will deposit. This means shorter paths will have stronger pheromone levels, so a higher chance it will be taken by the next ant.

It is important to avoid loops, this will cause a self-reinforcing effect where ants keep walking in loops because the pheromone levels favour this path. An elegant way of solving this problem is by giving the ants memory so they will not take a node they have already passed on their path.

To make sure eventually all the ants converge to the shortest paths, old (and longer) paths must be avoided. This is done by having pheromones evaporate over time. By doing this old paths eventually will no longer be taken. The evaporation is the same on each edge, for instance by multiplying the pheromone level on each edge with a certain value smaller than one.
Figure 1 shows a simple illustration of an ants algorithm. On the left figure, the first ants starts. The middle figure shows a snapshot in the middle of the program, the pheromones are still pretty equal, so the distribution of the ants is also equal. On the right figure, all the ants walk the shortest path, this is the end of the program.



Figuur 1: Simple illustration ant algorithm[1]

3 Method

This work sought a way to perform this algorithm in using CUDA. First we will go over the used data structures, next we will explain how the program works.

3.1 Data Structures

Each node is represented with an index. The source node is always node zero. The graph is unweighted, however the algorithm still works if the graph is weighted (with some minor changes). It does not matter if the graph is directed or not, because of the use of adjacency lists.

In this program several data structures where used. First we have the adjacency list. This list contains all the nodes adjacent to a certain node, meaning there is a direct edge between the two nodes. For the structure of the adjacency list, multiple options are possible.

We could use a matrix of the size $N * N$, with N being the amount of nodes. The matrix is then flattened into a single array. Indexing the element on row x and column y is done by: $matrix[x * N + y]$. The upside of this matrix is that it can be flattened, which makes for very fast access to an element in the array (if the x and y are known). The downside is that this matrix will be very sparse, since it is rare for a node to connect to all the other nodes. A separate *lengths*-array is used to indicate how much adjacent nodes each node has. So when an ant with index i has length l, this means that $array[i * N]$ until $array[i * N + l - 1]$ contain the adjacent nodes, $array[i * N + l]$ until $array[i * N + N - 1]$ contain some random non important value (for instance -1).

An improvement to the matrix above would be to first extract the longest adjacency list, and use that in the shape of the matrix. For instance if the node with the most neighbours has M neighbours, the shape of the adjacency matrix would be: $N * M$. This approach is used in the implementation for this assignment.

Another method is to use a non-flattened matrix, meaning an array with size N, with each element containing another array with the adjacency list for the node with the corresponding index. The upside of this method is that it is the most memory-efficient way, but it has slower memory access because the elements in the matrix can not directly be accessed (two memory references instead of one).

Next we have the pheromone matrix. This matrix contains the pheromone level on each edge. For instance if the edge between node x and node y have a pheromone level of 2, this is shown in the matrix as $pheromoneMatix[x][y] = 2$. The shape of the matrix is thus $N * N$ with N being the amount of nodes in the graph. The matrix is also flattened for easier faster memory access (so access to edge between x and y would be: $pheromoneMatrix[x * N + y] = 2$) . We could also represent the pheromones using a non-flattened matrix, but with the same downsides as for the adjacency lists. The pheromone matrix is initialized with each element equal to one.

Finally we have the ants themselves. They have a pretty simple structure, each ant has a index, a counter indicating the amount of nodes it has already passed (*nodeCounter*) and a list containing each passed node (starting from the sink to the current node).

3.2 The algorithm on the GPU

Each ant is represented by a different thread. This means all the threads can move in parallel. The flow of the program is given in Figure 3.

The ants are represented by two arrays: one array contains the *nodeCounter* of each ant, called *ants_counters*, this array has the size A which is the amount of ants. The other array is a $A * N$ array, called *ants_paths*, containing the current path of each ant[1]. The first node (so source node) of ant wit index i can be found at $ants_paths[i * A]$, the last node on its current path at $ants_paths[i * A + ants_counters[i]]$.

The ants move in steps, meaning first all the ants move, then we update all pheromone matrix, evaporate some of the pheromones, and repeat the process for a while. The duration of the loop is experimentally determend, we have no certain logic to determine how long it will take the ants to converge to the shortest path.

While moving the ants no synchronization is needed, each thread is the only one accessing the elements in the *ants_counters* and *ants_paths*. And threads are only reading from the pheromone matrix and the adjacency matrix in this function. The ant selects its next node based on a weighted probability function with the weights corresponding to the pheromone level on each edge divided by the total amount of pheromones on the outgoing edges. Of course only the edges to nodes adjacent to the current node are used.

After waiting on all the ants to move we go into the function where we spread the pheromones. Here we check each ant's last node, if this node is the sink, we update the pheromone levels on the edges in it's path. The amount of pheromones per path we add is equal to one divided by the sum of all the edges in the path, this results in more pheromones per edge when the edge belongs to a shorter path. When an ant has reached the sink, the ant is reset and runs again from source in the next iteration. This function is also done using threads, but now we need synchronization, because multiple threads might try to add values to the same element in the pheromone matrix. Because of this, we use an atomic addition (which is part of CUDA).

Once again we wait for all the threads to finish. Now we launch another kernel which runs over the pheromone matrix and multiplies each element with a certain constant (in our case 0,9). Again no synchronization is needed because each thread only accesses one element. This can be done by spawning N*N threads or by using stride.

After we run the above three functions in a loop for a while, most ants will start walking the shortest path. It is therefor important not to stop to early, we want to give the ants the time to converge to the shortest path. After we stop the program, we can extract the shortest path by looking at the pheromone matrix. The matrix will have changed in such a way that we can extract the shortest path by starting at the source, going to the node with the most pheromones on the edge between them, adding that node to our path and repeating this until we reach the sink. This function is partially done in parallel, finding the edge with the highest pheromone level for each node is done in parallel, this is saved to an array. After synchronizing thread zero uses these values to extract the shortest path.

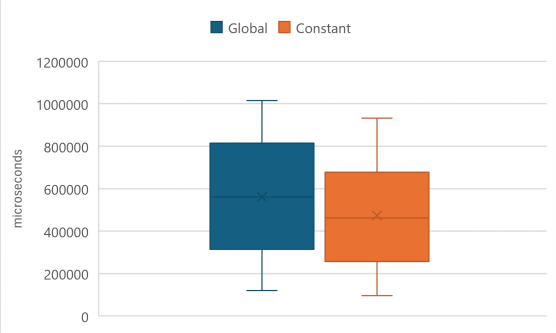
3.3 Optimizations

To make the program faster we can save the adjacency matrix and lengths array in constant memory, because it is only being read. Unfortunately the constant memory is only 64KB, this effectively bounds the size of the graph. Since we represent each node with an integer (4 bytes) this gives us 16384 places for nodes. If we use an adjacency matrix of the size $N * N$ this bounds N at 128 nodes (127 if the *lengths* array is also saved in constant memory). However with the methods discussed in Section 3.1 we can use a more memory-efficient matrix.

[1] we assume a ant will never walk a path longer than N nodes

A second (minor) optimization is by using shared memory when finding the shortest path, since our test graphs had less than 1024 nodes we could use this form of fast, on-chip memory to store the best adjacent node to go to based on pheromone level.

The comparison of constant memory versus global memory is given in Figure 2. We ran the algorithm 27 times with 3 warm up rounds, once for constant and once for global memory. As the figure shows, the constant memory is faster. The difference is however quite small because a lot of data structures (such as the ant arrays and pheromone matrix) still use global memory.



Figuur 2: Adjacency Matrix in Global versus Constant Memory

4 Improvements

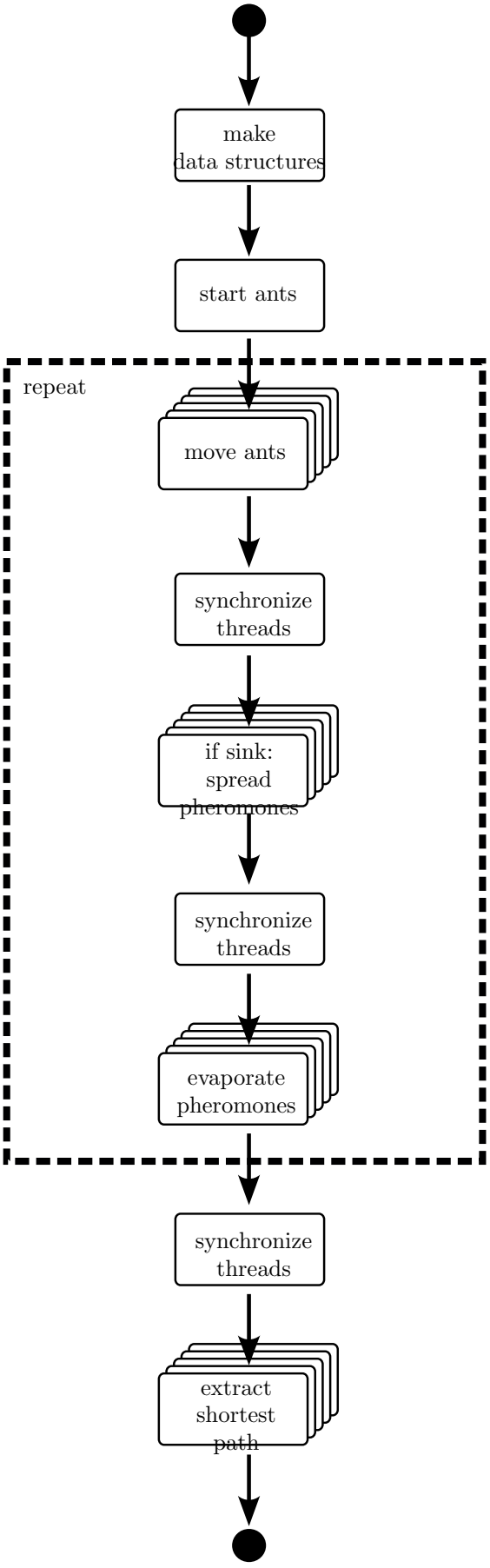
For now, my program only works with unweighted graphs, this can easily be changed taking the weight of an edge in account in the weighted probability function when choosing an edge. Also edges with a high cost can have their pheromones evaporate quicker.

The code itself can also be improved, it is not super efficiently written, and can thus be made quite a bit faster. Also a lot of parameters (such as most adjacent nodes) have to be hard coded right now which could be changed if the deadline was not so close by.

Finally another improvement would be by using dynamic parallelism, for instance when calculating the weights for each node in the probability function (when the ant is choosing a node). Unfortunately dynamic parallelism is not possible on my GPU, so I could not test the performance improvements.

Referenties

"The-operation-of-the-Ant-Colony-algorithm", researchgate. January 2019 Published. [Online]. Available: https://www.researchgate.net/figure/The-operation-of-the-Ant-Colony-algorithm-2-Methodology-Implementation-of-the-Ant-Colony_fig1_330697852. Accessed on: 5 may, 2024.



Figuur 3: Flow graph of program